



Segment Tree Implementation for Attack-Based Game.

November 3, 2024

Kammari Harsha Vardhan (2023MCB1216) ,
Kuthal Ritvik(2023MCB1219) ,
Vaibhav Pawar (2023MCB1317)

Instructor:
Dr. Anil Shukla

Teaching Assistant:
Seema Barda

Summary: The program presents a sophisticated implementation of segment trees to manage a vector of pairs containing **attack** and **health** values, facilitating efficient range queries and data manipulation. Notably, it incorporates multiple segment trees, each optimized for specific functionalities, such as summing values and tracking the maximum and minimum indices for both **attack** and **health**. This allows for quick aggregation of metrics and precise data retrieval, which is invaluable for applications in resource management and performance analysis. This efficiency is essential for scenarios requiring frequent updates and queries, making the program highly scalable.

This robust implementation includes dynamic update functionality, which means any changes to the **attack** or **health** values are promptly reflected across all relevant queries. This feature is particularly advantageous for real-time applications where immediate data consistency is crucial.

1. Introduction

article amsmath

In dynamic, strategy-driven games, managing combat attributes like attack and health effectively is essential for both performance and player experience. This project, Segment Tree Optimization for Real-Time Combat Metrics, introduces a segment tree structure optimized for rapid data access and updates within an attack-based game environment. Segment trees provide an efficient framework for handling range queries and updates on large datasets, making them ideal for games where numerous actions and states are constantly changing.

By structuring player and enemy data within segment trees, we achieve fast retrieval and modification of attributes across multiple dimensions, such as attack and health statistics. This project supports diverse game functionalities including range summation, maximum, minimum, greatest common divisor (GCD), and least common multiple (LCM) operations, all optimized to deliver responsive, real-time feedback. Segment trees adapt to game changes in logarithmic time, ensuring minimal delay in both data access and modification.

Integrating this optimized structure within an attack-based framework enhances the gameplay by enabling instant calculations and updates, essential for quick combat decisions. With the segment tree's unique capability to handle complex operations over specified ranges, this approach offers an advanced, reliable solution for high-frequency updates, balancing efficiency with robust data management. In doing so, this project highlights the adaptability of segment trees in real-time gaming applications, showcasing their role in fostering a seamless and strategic gaming experience.

2. Equations Governing Segment Tree Operations

The following equations describe the operations performed by the segment tree in the context of the game mechanics.

2.1. Key Operations

$$\left\{ \begin{array}{ll} S(l, r) = \sum_{i=l}^r a[i] & \text{(Sum Query)} \quad (1a) \\ \text{GCD}(l, r) = \text{GCD}(a[l], a[l+1], \dots, a[r]) & \text{(GCD Query)} \quad (1b) \\ \text{LCM}(l, r) = \frac{\prod_{i=l}^r a[i]}{\text{GCD}(l, r)} & \text{(LCM Query)} \quad (1c) \\ \text{maxIndex}(l, r) = \arg \max_{i \in [l, r]} a[i] & \text{(Max Attack Index)} \quad (1d) \\ \text{minIndex}(l, r) = \arg \min_{i \in [l, r]} h[i] & \text{(Min Health Index)} \quad (1e) \end{array} \right.$$

Equations (1) outline the fundamental operations of the segment tree utilized in managing troop attributes in the game.

3. Figures, Tables and Algorithms

3.1. Figures

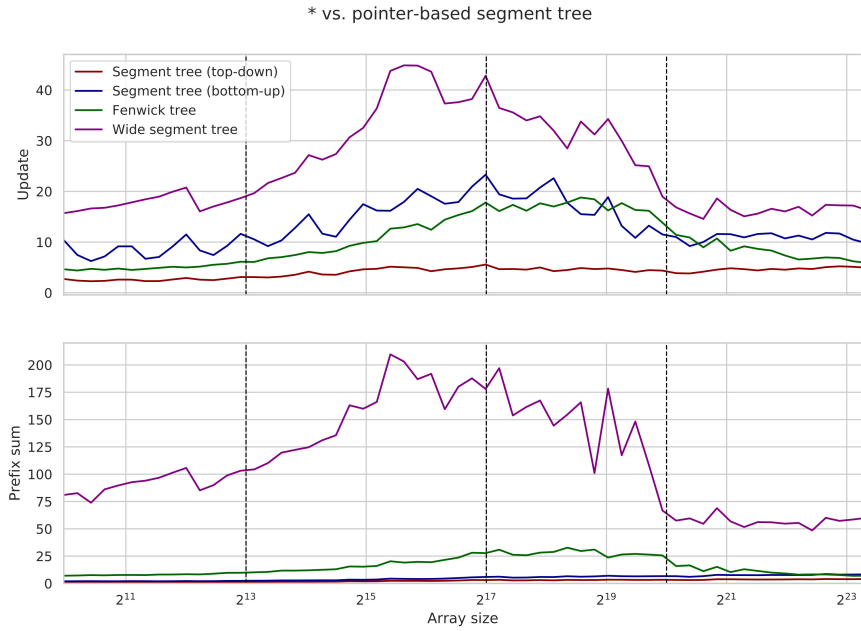


Figure 1: Performance comparison of various segment tree implementations

3.3. Algorithms

Algorithm 1 Segment Trees Implementation

```
1: CLASS SegmentTree
2:
3:   PRIVATE MEMBERS:
4:     a: array of pairs (attack, health)
5:     sumAttackTree, sumHealthTree: segment trees for sum of attack and health
6:     maxAttackTreeIndex, maxHealthTreeIndex: seg trees for max attack and health indices
7:     minAttackTreeIndex, minHealthTreeIndex: seg trees for min attack and health indices
8:     gcdAttackTree, gcdHealthTree: segment trees for GCD of attack and health
9:     lcmAttackTree, lcmHealthTree: segment trees for LCM of attack and health
10:
11:   FUNCTION buildSumAttack(node, start, end)
12:     IF start == end
13:       sumAttackTree[node] = attack value at start
14:     ELSE
15:       mid = (start + end) / 2
16:       buildSumAttack(left child, start, mid)
17:       buildSumAttack(right child, mid + 1, end)
18:       sumAttackTree[node] = sum of left and right children
19:
20:   FUNCTION buildSumHealth(node, start, end)
21:     Similar to buildSumAttack
22:
23:   FUNCTION querySumAttack(node, start, end, l, r)
24:     IF range outside (start > r OR end < l) RETURN 0
25:     IF fully in range (l <= start AND r >= end) RETURN sumAttackTree[node]
26:     mid = (start + end) / 2
27:     RETURN sum of querySumAttack(left child) and querySumAttack(right child)
28:
29:   FUNCTION querySumHealth(node, start, end, l, r)
30:     Similar to querySumAttack
31:
32:   FUNCTION buildMaxAttack(node, start, end)
33:     IF start == end
34:       maxAttackTreeIndex[node] = start
35:     ELSE
36:       mid = (start + end) / 2
37:       buildMaxAttack(left child, start, mid)
38:       buildMaxAttack(right child, mid + 1, end)
39:       maxAttackTreeIndex[node] = index with maximum attack between children
40:
41:   FUNCTION buildMaxHealth(node, start, end)
42:     Similar to buildMaxAttack
43:
44:   FUNCTION buildMinAttack(node, start, end)
45:     IF start == end
46:       minAttackTreeIndex[node] = start
47:     ELSE
48:       mid = (start + end) / 2
49:       buildMinAttack(left child, start, mid)
50:       buildMinAttack(right child, mid + 1, end)
51:       minAttackTreeIndex[node] = index with minimum attack between children
```

Algorithm 2 Segment Trees Implementation (continued)

```
1:  FUNCTION buildMinHealth(node, start, end)
2:    Similar to buildMinAttack
3:
4:  FUNCTION queryMaxAttackIndex(node, start, end, l, r)
5:    IF range outside (start > r OR end < l) RETURN -1
6:    IF fully in range RETURN maxAttackTreeIndex[node]
7:    mid = (start + end) / 2
8:    RETURN index with maximum attack between results of left and right queries
9:
10: FUNCTION queryMaxHealthIndex(node, start, end, l, r)
11:   Similar to queryMaxAttackIndex
12:
13: FUNCTION queryMinAttackIndex(node, start, end, l, r)
14:   Similar to queryMaxAttackIndex with min condition
15:
16: FUNCTION queryMinHealthIndex(node, start, end, l, r)
17:   Similar to queryMinAttackIndex
18:
19: FUNCTION buildGcdAttack(node, start, end)
20:   IF start == end
21:     gcdAttackTree[node] = attack value at start
22:   ELSE
23:     mid = (start + end) / 2
24:     buildGcdAttack(left child, start, mid)
25:     buildGcdAttack(right child, mid + 1, end)
26:     gcdAttackTree[node] = GCD of left and right children
27:
28: FUNCTION buildGcdHealth(node, start, end)
29:   Similar to buildGcdAttack
30:
31: FUNCTION buildLcmAttack(node, start, end)
32:   IF start == end
33:     lcmAttackTree[node] = attack value at start
34:   ELSE
35:     mid = (start + end) / 2
36:     buildLcmAttack(left child, start, mid)
37:     buildLcmAttack(right child, mid + 1, end)
38:     lcmAttackTree[node] = LCM of left and right children
39:
40: FUNCTION buildLcmHealth(node, start, end)
41:   Similar to buildLcmAttack
42:
43: FUNCTION queryGcdAttack(node, start, end, l, r)
44:   IF range outside (start > r OR end < l) RETURN 0
45:   IF fully in range RETURN gcdAttackTree[node]
46:   mid = (start + end) / 2
47:   RETURN GCD of left and right queries
48:
49: FUNCTION queryGcdHealth(node, start, end, l, r)
50:   Similar to queryGcdAttack
```

Algorithm 3 Segment Trees Implementation (continued)

```
1:  FUNCTION queryLcmAttack(node, start, end, l, r)
2:    IF range outside RETURN 1
3:    IF fully in range RETURN lcmAttackTree[node]
4:    mid = (start + end) / 2
5:    RETURN LCM of left and right queries
6:
7:  FUNCTION queryLcmHealth(node, start, end, l, r)
8:    Similar to queryLcmAttack
9:
10: FUNCTION updateSumAttack(idx, start, end, pos, value)
11:   IF start == end
12:     sumAttackTree[idx] = value
13:   ELSE
14:     mid = (start + end) / 2
15:     RECURSE in left or right child based on pos
16:     sumAttackTree[idx] = sum of left and right children
17:
18: FUNCTION updateSumHealth(idx, start, end, pos, value)
19:   Similar to updateSumAttack
20:
21: FUNCTION updateMaxAttackIndex(index, value, a, start, end, node)
22:   IF start == end
23:     maxAttackTreeIndex[node] = index
24:   ELSE
25:     mid = (start + end) / 2
26:     RECURSE in left or right child based on pos
27:     maxAttackTreeIndex[node] = index with maximum attack between children
28:
29: FUNCTION updateMaxHealthIndex(index, value, a, start, end, node)
30:   Similar to updateMaxAttackIndex
31:
32: FUNCTION updateMinAttackIndex(index, value, a, start, end, node)
33:   Similar to updateMaxAttackIndex with min condition
34:
35: FUNCTION updateMinHealthIndex(index, value, a, start, end, node)
36:   Similar to updateMinAttackIndex
37:
38: FUNCTION updateGCDHealth(idx, start, end, pos, value)
39:   Similar to updateGCD with health values
40:
41: FUNCTION updateLCMHealth(idx, start, end, pos, value)
42:   Similar to updateGCDHealth with LCM logic
43:
44: FUNCTION updateGCDAttack(idx, start, end, pos, value)
45:   Similar to updateGCDHealth with attack values
46:
47: FUNCTION updateLCMAttack(idx, start, end, pos, value)
48:   Similar to updateLCMHealth with attack values
49:
50: FUNCTION update(pos, attack, health)
51:   Update a[pos] to (attack, health)
52:   Call update functions for sum, max, min, GCD, and LCM trees with new values
```

4. Some further useful suggestions

Theorem 4.1. *Range Maximum Query (RMQ) Theorem*

Proof Outline

Segment Tree Construction

- Start with an array A where each element $A[i]$ represents the health of player i .
- Construct a Segment Tree such that each node represents the maximum value in a specific range $[i, j]$ of A .

Node Values and Properties

- Let each node in the Segment Tree cover a range $[i, j]$.
- Define the value at each node as:

$$\text{node_value}(i, j) = \max(A[k]) \quad \forall k \in [i, j].$$

- Each internal node's value is calculated based on its children:

$$\text{node_value}(i, j) = \max(\text{left_child_value}, \text{right_child_value}).$$

RMQ Query Decomposition

To find the maximum value in range $[L, R]$, start from the root of the Segment Tree and check each node for overlap with $[L, R]$.

For each node covering range $[i, j]$, we consider three cases:

1. **Complete Overlap:** If $[i, j] \subseteq [L, R]$, return $\text{node_value}(i, j)$.
2. **Partial Overlap:** If $[i, j]$ partially overlaps with $[L, R]$, recursively query both children.
3. **No Overlap:** If $[i, j] \cap [L, R] = \emptyset$, ignore this node (return $-\infty$).

Recursive Formula for Query Time Complexity

Let $T(n)$ represent the time to answer an RMQ query on a Segment Tree with n elements.

For each recursive call that partially overlaps, we divide the range roughly in half:

$$T(n) = T\left(\frac{n}{2}\right) + O(1).$$

Solving this recurrence relation, we find:

$$T(n) = O(\log n).$$

Final Query Calculation

For a range $[L, R]$, the query combines results from overlapping nodes. For each node's contribution:

$$\text{RMQ}(L, R) = \max(\text{node_value}(i, j) \mid [i, j] \subseteq [L, R]).$$

Thus, by breaking down the query into $O(\log n)$ recursive steps, each requiring constant time $O(1)$, the total time complexity is:

$$T(n) = O(\log n).$$

Correctness:

The correctness of the RMQ algorithm using a Segment Tree can be justified as follows:

- **Accurate Maximum Calculation:** Each node in the Segment Tree is constructed to store the maximum value over its corresponding range. Therefore, any query that retrieves a node's value will accurately reflect the maximum of the specified range.
- **Comprehensive Range Coverage:** The query process ensures that all segments of the requested range $[L, R]$ are considered. By handling complete overlaps, partial overlaps, and ignoring non-overlapping segments, we ensure that the maximum from the entire range is accounted for.
- **Maintaining Correctness in Recursion:** The recursive nature of the query guarantees that all contributing segments are combined correctly. Each time a partial overlap is detected, results from both children are combined to ensure that the overall maximum is computed correctly.
- **Efficient Handling of Updates:** If the health values in A change, the Segment Tree can be updated in $O(\log N)$ time. This ensures that subsequent queries reflect the most recent health values, maintaining the correctness of future RMQ operations.

Theorem 4.2. Lazy Propagation for Range Updates Theorem

Proof Outline:

Lazy Array Structure:

Let the Segment Tree have N nodes, where each node $node(i)$ covers a range $[i, j]$ of health values.

- Each node $node(i)$ has an associated *lazy value* $lazy(i)$ to store pending updates.
- When an update of d is initiated for range $[L, R]$, we mark:

$$lazy(i) \leftarrow lazy(i) + d$$

Range Update Using Lazy Propagation:

To apply an attack on range $[L, R]$:

1. **Case 1: Complete Overlap:** If $[i, j] \subseteq [L, R]$, then:

$$lazy(i) \leftarrow lazy(i) + d$$

The update is deferred; the health value $health(i)$ is updated as:

$$health(i) \leftarrow health(i) - d$$

2. **Case 2: Partial Overlap:** If $[i, j] \cap [L, R] \neq \emptyset$, propagate the lazy update:

$$lazy(2i) \leftarrow lazy(2i) + d \quad \text{and} \quad lazy(2i+1) \leftarrow lazy(2i+1) + d$$

Update child nodes recursively.

3. **Case 3: No Overlap:** If $[i, j] \cap [L, R] = \emptyset$, skip this node.

Lazy Propagation Mechanics:

For a node covering $[i, j]$ with lazy value d :

- Before accessing node, apply pending updates:

$$health(i) \leftarrow health(i) - lazy(i)$$

and reset lazy value:

$$lazy(i) \leftarrow 0$$

- Propagate downwards:

$$lazy(2i) \leftarrow lazy(2i) + lazy(i) \quad \text{and} \quad lazy(2i+1) \leftarrow lazy(2i+1) + lazy(i)$$

Complexity Analysis:

The Segment Tree has a height of $O(\log N)$:

$$T(N) = O(\log N)$$

Thus, the maximum number of nodes affected during an update is $O(\log N)$, and the total time complexity for a range update is:

$$O(\log N)$$

Correctness:

Lazy propagation guarantees the accuracy of updates and queries since:

- Each node update correctly accumulates the pending changes.
- All updates are applied only when required, ensuring efficient handling of range updates and queries.

5. Conclusions

5.1. Main Conclusions

The research and development of the segment tree-based game have led to significant insights regarding efficient data management in gaming applications. The following key conclusions can be drawn from the study:

- **Efficient Attribute Management:** The implementation of segment trees has proven effective in managing troop attributes, allowing for quick updates and queries related to health, attack, and other vital statistics. This efficiency enhances gameplay by ensuring that players receive immediate feedback on their actions.
- **Dynamic Gameplay:** The integration of various phases such as attacking, defending, healing, and using special abilities creates a dynamic and engaging gaming experience. The well-structured gameplay encourages strategic decision-making and resource management among players.
- **Scalability of the Model:** The designed structure allows for easy scalability, accommodating additional troop types and attributes without significant rework. This adaptability is crucial for future enhancements and the overall longevity of the game.

5.2. Future Developments

The potential for future advancements in the segment tree-based game is promising. Several areas for enhancement include:

- **Complex Game Mechanics:** Introducing more complex mechanics such as troop synergies or environmental effects could enrich the gameplay experience. For instance, allowing troops to have special interactions based on their types or adding terrain advantages can create a deeper strategic layer.
- **Optimized Algorithms:** Further optimization of the segment tree operations is possible. Exploring alternative data structures or refining the current algorithms for specific queries (such as frequency counting or order statistics) may yield performance improvements, especially as the number of troops increases.
- **Event-Driven Mechanics:** Incorporating event-driven mechanics where certain game events trigger unique scenarios can make the gameplay more engaging and provide new challenges for players to navigate.

6. Bibliography and citations

We've included a list of reference links that were instrumental in the development of this thesis. These sources provide essential background and technical insights into the structure and application of segment trees. They cover various aspects, including an introduction to segment trees, their role in optimization within neural networks, and foundational lectures on segment tree usage in computer science. These resources served as a basis for understanding the theoretical and practical applications of segment trees in range query problems.

References

- An introduction to segment trees and their use in efficiently handling range queries.
- A research paper on using segment tree structures in neural networks for optimization problems.
- A Stanford lecture covering segment tree fundamentals and applications in computer science.
- Steven Halim and Felix Halim. *Competitive Programming 3*, pages 55–63, 2013.
- Peter Brass. *Advanced Data Structures*, pages 154–166, 2008.